

Studentu pazymiu skaiciuotuvas

v1.5

Generated by Doxygen 1.13.2

1 ObjPirmaUzduotis	1
1.0.1 Duomenų išvestis	1
1.1 Testavimo sistemos parametrai:	1
1.2 Testai su klasėmis (naudojant vector)	1
1.3 Testai su struktūromis (naudojant vector)	2
1.4 Flag testai:	2
1.5 O1 flag, .exe failo dydis: 309 KB	2
1.6 O2 flag, .exe failo dydis: 273 KB	2
1.7 O3 flag, .exe failo dydis: 318 KB	3
2 Hierarchical Index	5
2.1 Class Hierarchy	5
3 Class Index	7
3.1 Class List	7
4 File Index	9
4.1 File List	9
5 Class Documentation	11
5.1 Studentas Class Reference	11
5.1.1 Member Function Documentation	12
5.1.1.1 spausdinti()	12
5.2 Zmogus Class Reference	12
6 File Documentation	13
6.1 funkcijos.h	13
6.2 mano_lib.h	18
Index	19

Chapter 1

ObjPirmaUzduotis

Projekto naudojimo instrukcija: Atidarykite projekto aplanką ir paleiskite program.exe

Arba

1) Atidarykite terminalą projekto direktorijoje (cd C:"projekto direktorija") 2) Sukurkite build direktoriją (mkdir build cd build) 3) Paleiskite CMake, norint sugeneruoti build failus (cmake ..) 4) Kompiliuokite projektą (cmake --build .) 5) Paleiskite failą (.ObjCppProject.exe)

@section autotoc_md2 Perdengtų metodų aprašymas @subsection autotoc_md3 Duomenų įvestis

Programoje yra keli būdai įvesti duomenis apie studentus:

1. **Rankiniu būdu**: - Naudojant `<tt>istream\& operator\>\>(istream\& in, Studentas\& studentas)</tt>` metodą, galima įvesti studento vardą, pavardę, pažymius ir egzamino rezultatą rankiniu būdu per konsolę. - Pavyzdys: `@icode Jonas Jonaitis 8 9 10 7 6 5 @endcode` Čia paskutinis skaičius (5) yra egzamino rezultatas, o likę skaičiai – pažymiai.
2. **Automatiniu būdu**: - Naudojant funkciją, galima sugeneruoti atsitiktinius studentų duomenis. Ši funkcija priima studentų skaičių ir konteinerį, į kurį bus įrašyti sugeneruoti studentai. - Pavyzdys: `@icode{cpp} vector<Studentas> studentai; GeneruotiStudentus(100, studentai); @endcode`
3. **Iš failo**: - Naudojant funkciją, galima nuskaityti studentų duomenis iš failo. Failas turi būti tinkamai suformatuotas (pirmoje eilutėje – antraštės, o toliau – studentų duomenys). - Pavyzdys: `@icode{cpp} vector<Studentas> studentai; NuskaitytiStudentusIsFailo("studentai1000.txt", studentai);`

1.0.1 Duomenų išvestis

Programoje taip pat yra keli būdai išvesti duomenis apie studentus:

1. Į ekraną
 2. Į failą
-

1.1 Testavimo sistemos parametrai:

Processor 12th Gen Intel(R) Core(TM) i5-1235U, 1300 Mhz, 10 Core(s), 12 Logical Processor(s)
Installed Physical Memory (RAM) 16.0 GB
SSD 512 GB NVMe Micron 2400 MTFDKBA512QFM

1.2 Testai su klasėmis (naudojant vector)

studentai100000.txt Studentu nuskaitymas iš failo užtruko: 3689 ms Studentu rikiavimas užtruko: 158 ms Studentu skaidymas ir irasymas į failus užtruko: 2043 ms Iš viso užtruko: 8654 ms
studentai1000000.txt Studentu nuskaitymas iš failo užtruko: 20653 ms Studentu rikiavimas užtruko: 919 ms Studentu skaidymas ir irasymas į failus užtruko: 14782 ms Iš viso užtruko: 42303 ms
studentai1000000.txt Studentu nuskaitymas iš failo užtruko: 3414 ms Studentu rikiavimas užtruko: 157 ms Studentu skaidymas ir irasymas į failus užtruko: 2561 ms Iš viso užtruko: 7009 ms

studentai1000000.txt Studentu nuskaitymas is failo uztruko: 21611 ms Studentu rikiavimas uztruko: 904 ms Studentu skaidymas ir irasymas i failus uztruko: 14855 ms Is viso uztruko: 42616 ms
studentai100000.txt Studentu nuskaitymas is failo uztruko: 3829 ms Studentu rikiavimas uztruko: 187 ms Studentu skaidymas ir irasymas i failus uztruko: 2722 ms Is viso uztruko: 7787 ms
studentai1000000.txt Studentu nuskaitymas is failo uztruko: 19429 ms Studentu rikiavimas uztruko: 820 ms Studentu skaidymas ir irasymas i failus uztruko: 15109 ms Is viso uztruko: 40092 ms

1.3 Testai su struktūromis (naudojant vector)

studentai100000.txt Studentu nuskaitymas is failo uztruko: 3392 ms Studentu rikiavimas uztruko: 159 ms Studentu skaidymas ir irasymas i failus uztruko: 1433 ms Is viso uztruko: 6611 ms
studentai1000000.txt Studentu nuskaitymas is failo uztruko: 19578 ms Studentu rikiavimas uztruko: 783 ms Studentu skaidymas ir irasymas i failus uztruko: 8246 ms Is viso uztruko: 36521 ms
studentai100000.txt Studentu nuskaitymas is failo uztruko: 3958 ms Studentu rikiavimas uztruko: 194 ms Studentu skaidymas ir irasymas i failus uztruko: 1645 ms Is viso uztruko: 7404 ms
studentai1000000.txt Studentu nuskaitymas is failo uztruko: 20860 ms Studentu rikiavimas uztruko: 768 ms Studentu skaidymas ir irasymas i failus uztruko: 8065 ms Is viso uztruko: 37336 ms
studentai100000.txt Studentu nuskaitymas is failo uztruko: 3283 ms Studentu rikiavimas uztruko: 133 ms Studentu skaidymas ir irasymas i failus uztruko: 1328 ms Is viso uztruko: 6050 ms
studentai1000000.txt Studentu nuskaitymas is failo uztruko: 20787 ms Studentu rikiavimas uztruko: 938 ms Studentu skaidymas ir irasymas i failus uztruko: 8911 ms Is viso uztruko: 38931 ms

1.4 Flag testai:

1.5 O1 flag, .exe failo dydis: 309 KB

studentai100000.txt Studentu nuskaitymas is failo uztruko: 261 ms Studentu rikiavimas uztruko: 31 ms Studentu skaidymas ir irasymas i failus uztruko: 1184 ms Is viso uztruko: 3311 ms
studentai1000000.txt Studentu nuskaitymas is failo uztruko: 1393 ms Studentu rikiavimas uztruko: 158 ms Studentu skaidymas ir irasymas i failus uztruko: 9316 ms Is viso uztruko: 13442 ms
studentai100000.txt Studentu nuskaitymas is failo uztruko: 319 ms Studentu rikiavimas uztruko: 22 ms Studentu skaidymas ir irasymas i failus uztruko: 1481 ms Is viso uztruko: 2275 ms
studentai1000000.txt Studentu nuskaitymas is failo uztruko: 1406 ms Studentu rikiavimas uztruko: 180 ms Studentu skaidymas ir irasymas i failus uztruko: 9146 ms Is viso uztruko: 13562 ms
studentai100000.txt Studentu nuskaitymas is failo uztruko: 238 ms Studentu rikiavimas uztruko: 22 ms Studentu skaidymas ir irasymas i failus uztruko: 1150 ms Is viso uztruko: 1909 ms
studentai1000000.txt Studentu nuskaitymas is failo uztruko: 1671 ms Studentu rikiavimas uztruko: 208 ms Studentu skaidymas ir irasymas i failus uztruko: 9555 ms Is viso uztruko: 15102 ms

1.6 O2 flag, .exe failo dydis: 273 KB

studentai100000.txt Studentu nuskaitymas is failo uztruko: 278 ms Studentu rikiavimas uztruko: 31 ms Studentu skaidymas ir irasymas i failus uztruko: 1210 ms Is viso uztruko: 3395 ms
studentai1000000.txt Studentu nuskaitymas is failo uztruko: 1385 ms Studentu rikiavimas uztruko: 217 ms Studentu skaidymas ir irasymas i failus uztruko: 9109 ms Is viso uztruko: 13341 ms
studentai100000.txt Studentu nuskaitymas is failo uztruko: 239 ms Studentu rikiavimas uztruko: 31 ms Studentu skaidymas ir irasymas i failus uztruko: 1452 ms Is viso uztruko: 2166 ms
studentai1000000.txt Studentu nuskaitymas is failo uztruko: 1384 ms Studentu rikiavimas uztruko: 202 ms Studentu skaidymas ir irasymas i failus uztruko: 8809 ms Is viso uztruko: 12908 ms
studentai100000.txt Studentu nuskaitymas is failo uztruko: 247 ms Studentu rikiavimas uztruko: 33 ms Studentu skaidymas ir irasymas i failus uztruko: 1376 ms Is viso uztruko: 2131 ms
studentai1000000.txt Studentu nuskaitymas is failo uztruko: 1365 ms Studentu rikiavimas uztruko: 198 ms Studentu skaidymas ir irasymas i failus uztruko: 8973 ms Is viso uztruko: 13254 ms

1.7 O3 flag, .exe failo dydis: 318 KB

studentai100000.txt Studentu nuskaitymas is failo uztruko: 240 ms Studentu rikiavimas uztruko: 24 ms Studentu skaidymas ir irasymas i failus uztruko: 1286 ms Is viso uztruko: 3045 ms
studentai1000000.txt Studentu nuskaitymas is failo uztruko: 1463 ms Studentu rikiavimas uztruko: 293 ms Studentu skaidymas ir irasymas i failus uztruko: 9018 ms Is viso uztruko: 13703 ms
studentai100000.txt Studentu nuskaitymas is failo uztruko: 251 ms Studentu rikiavimas uztruko: 24 ms Studentu skaidymas ir irasymas i failus uztruko: 1298 ms Is viso uztruko: 2056 ms
studentai1000000.txt Studentu nuskaitymas is failo uztruko: 1362 ms Studentu rikiavimas uztruko: 179 ms Studentu skaidymas ir irasymas i failus uztruko: 8984 ms Is viso uztruko: 13076 ms
studentai100000.txt Studentu nuskaitymas is failo uztruko: 249 ms Studentu rikiavimas uztruko: 23 ms Studentu skaidymas ir irasymas i failus uztruko: 1282 ms Is viso uztruko: 2034 ms
studentai1000000.txt Studentu nuskaitymas is failo uztruko: 1403 ms Studentu rikiavimas uztruko: 179 ms Studentu skaidymas ir irasymas i failus uztruko: 8881 ms Is viso uztruko: 13072 ms

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Zmogus	12
Studentas	11

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas	11
Zmogus	12

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

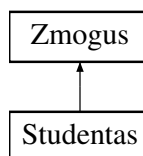
funkcijos.h	13
mano_lib.h	18

Chapter 5

Class Documentation

5.1 Studentas Class Reference

Inheritance diagram for Studentas:



Public Member Functions

- **Studentas** (const string &vardas, const string &pavarde, const vector< int > &pazymiai, int egzaminas)
- **Studentas** (const [Studentas](#) &other)
- **Studentas** ([Studentas](#) &&other) noexcept
- [Studentas](#) & **operator=** (const [Studentas](#) &other)
- [Studentas](#) & **operator=** ([Studentas](#) &&other) noexcept
- string **getVardas** () const
- string **getPavarde** () const
- vector< int > **getPazymiai** () const
- vector< int > & **getPazymiaiRef** ()
- int **getEgzaminas** () const
- float **getGalutinis** () const
- void **setVardas** (const string &v)
- void **setPavarde** (const string &p)
- void **setPazymiai** (const vector< int > &p)
- void **setEgzaminas** (int e)
- void **setGalutinis** (float g)
- void **addPazymys** (int pazymys)
- void [spausdinti](#) () const override

Public Member Functions inherited from [Zmogus](#)

- **Zmogus** (const string &v, const string &p)
- string **getVardas** () const
- string **getPavarde** () const
- void **setVardas** (const string &v)
- void **setPavarde** (const string &p)

Friends

- `istream & operator>> (istream &in, Studentas &studentas)`
- `ostream & operator<< (ostream &out, const Studentas &studentas)`

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- string **vardas**
- string **pavarde**

5.1.1 Member Function Documentation

5.1.1.1 `spausdinti()`

`void Studentas::spausdinti () const [inline], [override], [virtual]`

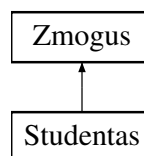
Implements [Zmogus](#).

The documentation for this class was generated from the following file:

- `funkcijos.h`

5.2 Zmogus Class Reference

Inheritance diagram for Zmogus:



Public Member Functions

- **Zmogus** (const string &v, const string &p)
- virtual void **spausdinti** () const =0
- string **getVardas** () const
- string **getPavarde** () const
- void **setVardas** (const string &v)
- void **setPavarde** (const string &p)

Protected Attributes

- string **vardas**
- string **pavarde**

The documentation for this class was generated from the following file:

- `funkcijos.h`

Chapter 6

File Documentation

6.1 funkcijos.h

```
00001 #include "mano_lib.h"
00002
00003 void GeneruotiPazymius(int pazymiuSk, vector<int>& pazymiai);
00004 float Vidurkis(vector<int> pazymiai);
00005 float Mediana(vector<int> pazymiai);
00006
00007 // Abstrakti bazinė klasė
00008 class Zmogus {
00009     protected:
00010         string vardas;
00011         string pavarde;
00012
00013     public:
00014         Zmogus() : vardas(""), pavarde("") {}
00015         Zmogus(const string& v, const string& p) : vardas(v), pavarde(p) {}
00016         virtual ~Zmogus() = default;
00017
00018         // virtuali funkcija kad klase butu abstarkti
00019         virtual void spausdinti() const = 0;
00020
00021         // Getteriai
00022         string getVardas() const { return vardas; }
00023         string getPavarde() const { return pavarde; }
00024
00025         // Setteriai
00026         void setVardas(const string& v) { vardas = v; }
00027         void setPavarde(const string& p) { pavarde = p; }
00028     };
00029
00030 class Studentas: public Zmogus {
00031     private:
00032
00033         vector<int> pazymiai;
00034         int egzaminas;
00035         float galutinis;
00036
00037     public:
00038         // Constructors
00039         Studentas() : egzaminas(0), galutinis(0.0f) {} // Default constructor
00040         Studentas(const string& vardas, const string& pavarde, const vector<int>& pazymiai, int
egzaminas)
00041             : Zmogus(vardas, pavarde), pazymiai(pazymiai), egzaminas(egzaminas), galutinis(0.0f) {}
00042
00043
00044         // Destructor
00045         ~Studentas() {
00046             vardas.clear();
00047             pavarde.clear();
00048             pazymiai.clear();
00049             egzaminas = 0;
00050             galutinis = 0.0f;
00051         }
00052
00053         // Copy constructor
00054         Studentas(const Studentas& other)
00055             : Zmogus(other.vardas, other.pavarde), pazymiai(other.pazymiai),
00056               egzaminas(other.egzaminas), galutinis(other.galutinis) {}
00057         // Move constructor
00058         Studentas(Studentas&& other) noexcept
00059             : Zmogus(move(other.vardas), move(other.pavarde), pazymiai(move(other.pazymiai)),
00060               egzaminas(other.egzaminas), galutinis(other.galutinis) {}
```

```

00061         other.egzaminas = 0;
00062         other.galutinis = 0.0f;
00063     }
00064
00065     // Copy assignment operator
00066     Studentas& operator=(const Studentas& other) {
00067         if (this == &other) return *this; // Self-assignment check
00068         vardas = other.vardas;
00069         pavarde = other.pavarde;
00070         pazymiai = other.pazymiai;
00071         egzaminas = other.egzaminas;
00072         galutinis = other.galutinis;
00073         return *this;
00074     }
00075
00076     // Move assignment operator
00077     Studentas& operator=(Studentas&& other) noexcept {
00078         if (this == &other) return *this; // Self-assignment check
00079         vardas = move(other.vardas);
00080         pavarde = move(other.pavarde);
00081         pazymiai = move(other.pazymiai);
00082         egzaminas = other.egzaminas;
00083         galutinis = other.galutinis;
00084         other.egzaminas = 0;
00085         other.galutinis = 0.0f;
00086         return *this;
00087     }
00088
00089     // Getters
00090     string getVardas() const { return vardas; }
00091     string getPavarde() const { return pavarde; }
00092     vector<int> getPazymiai() const { return pazymiai; }
00093     vector<int>& getPazymiaiRef() { return pazymiai; }
00094     int getEgzaminas() const { return egzaminas; }
00095     float getGalutinis() const { return galutinis; }
00096
00097     // Setters
00098     void setVardas(const string& v) { vardas = v; }
00099     void setPavarde(const string& p) { pavarde = p; }
00100     void setPazymiai(const vector<int>& p) { pazymiai = p; }
00101     void setEgzaminas(int e) { egzaminas = e; }
00102     void setGalutinis(float g) { galutinis = g; }
00103
00104     void addPazymys(int pazymys) {
00105         pazymiai.push_back(pazymys);
00106     }
00107
00108     // Implementuojame abstrakčią funkciją
00109     void spausdinti() const override {
00110         cout << left << setw(15) << vardas << setw(20) << pavarde;
00111
00112         for (const auto& pazymys : pazymiai) {
00113             cout << setw(10) << pazymys;
00114         }
00115
00116         cout << setw(10) << egzaminas << fixed << setprecision(2) << galutinis << endl;
00117     }
00118
00119     // Input operator
00120     friend istream& operator>(istream& in, Studentas& studentas) {
00121         studentas.pazymiai.clear(); // Clear previous data
00122
00123         // Read vardas and pavarde
00124         in >> studentas.vardas >> studentas.pavarde;
00125         if (in.fail()) return in; // Return if reading failed
00126
00127         // Read grades and exam score
00128         int pazymys;
00129         vector<int> tempPazymiai;
00130         while (in >> pazymys) {
00131             tempPazymiai.push_back(pazymys);
00132         }
00133
00134         // Handle the case where no grades or exam score are provided
00135         if (tempPazymiai.empty()) {
00136             in.clear(); // Clear fail state
00137             return in;
00138         }
00139
00140         // Treat the last integer as the exam score
00141         studentas.egzaminas = tempPazymiai.back();
00142         tempPazymiai.pop_back(); // Remove the exam score from the grades
00143         studentas.pazymiai = tempPazymiai;
00144
00145         // Clear the stream state for the next read
00146         in.clear(); // Clear fail state but do not ignore the next line
00147     }

```

```

00148         return in;
00149     }
00150
00151     // Output operator
00152     friend ostream& operator<<(ostream& out, const Studentas& studentas) {
00153         out << left << setw(15) << studentas.vardas
00154             << setw(20) << studentas.pavarde;
00155         for (const auto& pazymys : studentas.pazymiai) {
00156             out << setw(10) << pazymys;
00157         }
00158         out << setw(10) << studentas.egzaminas
00159             << fixed << setprecision(2) << studentas.galutinis;
00160         return out;
00161     }
00162 };
00163
00164
00165 template <typename Container>
00166 void GeneruotiStudentus(int studentuSk, Container& studentai) {
00167     if (studentuSk <= 0) {
00168         throw invalid_argument("Studentu skaicius turi buti teigiamas");
00169     }
00170     vector<string> vardai = {"Jonas", "Petras", "Antanas", "Tomas", "Marius"};
00171     vector<string> pavardes = {"Jonaitis", "Petraitis", "Antanaitis", "Tomaitis", "Maraitis"};
00172
00173     for (int i = 0; i < studentuSk; i++) {
00174         Studentas studentas;
00175         studentas.setVardas(vardai[rand() % vardai.size()]);
00176         studentas.setPavarde(pavardes[rand() % pavardes.size()]);
00177
00178         vector<int> pazymiai;
00179         GeneruotiPazymius(rand() % 10 + 1, pazymiai);
00180         studentas.setPazymiai(pazymiai);
00181
00182         studentas.setEgzaminas(rand() % 10 + 1);
00183         studentai.push_back(studentas);
00184     }
00185 }
00186
00187 template <typename Container>
00188 void NuskaitytiStudentusIsFailo(string failas, Container& studentai) {
00189     auto start = high_resolution_clock::now();
00190
00191     ifstream in(failas);
00192     if (!in.is_open()) {
00193         throw runtime_error("Nepavyko atidaryti failo");
00194     }
00195
00196     cout << "Reading file: " << failas << endl;
00197     string line;
00198     getline(in, line); // Skip the header line
00199
00200     if constexpr (is_same<Container, vector<Studentas>::value>) {
00201         studentai.reserve(10000000); // Reserve memory for vector or deque
00202     }
00203
00204     int studentCount = 0; // Counter to track the number of students read
00205     while (true) {
00206         Studentas studentas;
00207         in >> studentas; // Use input operator
00208         if (in.fail()) {
00209             break; // Stop reading if the stream is in a fail state
00210         }
00211         studentai.push_back(std::move(studentas)); // Use move semantics
00212         studentCount++;
00213     }
00214
00215     in.close();
00216     auto end = high_resolution_clock::now();
00217     auto duration = duration_cast<milliseconds>(end - start);
00218     cout << "Studentu nuskaitymas is failo uztruko: " << duration.count() << " ms" << endl;
00219
00220     // Print the total number of students read
00221     cout << "Total students read: " << studentCount << endl;
00222 }
00223
00224 template <typename Container>
00225 void RikiuotiStudentus(Container& studentai, int pasirinkimas) {
00226     auto start = high_resolution_clock::now();
00227
00228     switch (pasirinkimas) {
00229         case 0:
00230             // nerikiuoti
00231             break;
00232         case 1:
00233             if constexpr (is_same<Container, list<Studentas>::value>) {
00234                 studentai.sort([](const Studentas& a, const Studentas& b) {
00235                     return a.getVardas() < b.getVardas();
00236                 });
00237             }
00238     }
00239 }

```

```

00235         });
00236     } else {
00237         sort(studentai.begin(), studentai.end(), [](const Studentas& a, const Studentas& b) {
00238             return a.getVardas() < b.getVardas();
00239         });
00240     }
00241     break;
00242 case 2:
00243     if constexpr (is_same<Container, list<Studentas>>::value) {
00244         studentai.sort([](const Studentas& a, const Studentas& b) {
00245             return a.getPavarde() < b.getPavarde();
00246         });
00247     } else {
00248         sort(studentai.begin(), studentai.end(), [](const Studentas& a, const Studentas& b) {
00249             return a.getPavarde() < b.getPavarde();
00250         });
00251     }
00252     break;
00253 case 3:
00254     if constexpr (is_same<Container, list<Studentas>>::value) {
00255         studentai.sort([](const Studentas& a, const Studentas& b) {
00256             return a.getGalutinis() < b.getGalutinis();
00257         });
00258     } else {
00259         sort(studentai.begin(), studentai.end(), [](const Studentas& a, const Studentas& b) {
00260             return a.getGalutinis() < b.getGalutinis();
00261         });
00262     }
00263     break;
00264 case 4:
00265     if constexpr (is_same<Container, list<Studentas>>::value) {
00266         studentai.sort([](const Studentas& a, const Studentas& b) {
00267             return a.getGalutinis() > b.getGalutinis();
00268         });
00269     } else {
00270         sort(studentai.begin(), studentai.end(), [](const Studentas& a, const Studentas& b) {
00271             return a.getGalutinis() > b.getGalutinis();
00272         });
00273     }
00274     break;
00275 default:
00276     throw invalid_argument("Neteisingas rikiavimo pasirinkimas");
00277 }
00278
00279 auto end = high_resolution_clock::now();
00280 auto duration = duration_cast<milliseconds>(end - start);
00281 cout << "Studentu rikiavimas uztruko: " << duration.count() << " ms" << endl;
00282 }
00283
00284 template <typename Container>
00285 void SkaiciuotiGalutini(Container& studentai, bool vid) {
00286     for (auto& studentas : studentai) {
00287         if (vid) {
00288             studentas.setGalutinis(0.4 * Vidurkis(studentas.getPazymiai()) + 0.6 *
00289 studentas.getEgzaminas());
00290         } else {
00291             studentas.setGalutinis(0.4 * Mediana(studentas.getPazymiai()) + 0.6 *
00292 studentas.getEgzaminas());
00293         }
00294     }
00295 }
00296
00297 template <typename Container>
00298 void StudentuAtskirimas(Container& studentai) {
00299     ofstream outVargsiukai("stud_b.txt");
00300     ofstream outKieti("stud_g.txt");
00301
00302     if (!outVargsiukai.is_open() || !outKieti.is_open()) {
00303         throw runtime_error("Nepavyko atidaryti failo");
00304     }
00305
00306     vector<Studentas> vargsiukai;
00307     vector<Studentas> kietis;
00308
00309     auto start = high_resolution_clock::now();
00310
00311     // Separate students into two vectors
00312     for (const auto& studentas : studentai) {
00313         if (studentas.getGalutinis() < 5) {
00314             vargsiukai.push_back(studentas);
00315         } else {
00316             kietis.push_back(studentas);
00317         }
00318     }
00319
00320     auto end = high_resolution_clock::now();
00321     auto duration = duration_cast<milliseconds>(end - start);

```

```

00320     cout << "Studentu suskirstymas uztruko: " << duration.count() << " ms" << endl;
00321
00322     auto start2 = high_resolution_clock::now();
00323
00324     // Write "vargsiukai" to file
00325     outVargsiukai << left << setw(15) << "Vardas" << setw(20) << "Pavarde";
00326     if (!vargsiukai.empty()) {
00327         for (size_t i = 1; i <= vargsiukai[0].getPazymiai().size(); i++) {
00328             outVargsiukai << setw(10) << ("ND " + to_string(i));
00329         }
00330     }
00331     outVargsiukai << setw(10) << "Egz." << endl;
00332
00333     for (const auto& studentas : vargsiukai) {
00334         outVargsiukai << left << setw(15) << studentas.getVardas()
00335             << setw(20) << studentas.getPavarde();
00336         for (const auto& pazymys : studentas.getPazymiai()) {
00337             outVargsiukai << setw(10) << pazymys;
00338         }
00339         outVargsiukai << setw(10) << studentas.getEgzaminas() << endl;
00340     }
00341
00342     // Write "kieti" to file
00343     outKieti << left << setw(15) << "Vardas" << setw(20) << "Pavarde";
00344     if (!kieti.empty()) {
00345         for (size_t i = 1; i <= kieti[0].getPazymiai().size(); i++) {
00346             outKieti << setw(10) << ("ND " + to_string(i));
00347         }
00348     }
00349     outKieti << setw(10) << "Egz." << endl;
00350
00351     for (const auto& studentas : kieti) {
00352         outKieti << left << setw(15) << studentas.getVardas()
00353             << setw(20) << studentas.getPavarde();
00354         for (const auto& pazymys : studentas.getPazymiai()) {
00355             outKieti << setw(10) << pazymys;
00356         }
00357         outKieti << setw(10) << studentas.getEgzaminas() << endl;
00358     }
00359
00360     auto end2 = high_resolution_clock::now();
00361     auto duration2 = duration_cast<milliseconds>(end2 - start2);
00362     cout << "Studentu irasymas i failus uztruko: " << duration2.count() << " ms" << endl;
00363
00364     outVargsiukai.close();
00365     outKieti.close();
00366 }
00367
00368 template <typename Container>
00369 void SkaidytiStudentus3Strategija(Container& studentai) {
00370     auto start = high_resolution_clock::now();
00371     Container vargsiukai;
00372     // Naudojame std::partition, kad "vargšiukai" būtų konteinerio gale
00373     auto it = std::partition(studentai.begin(), studentai.end(), [](const Studentas& studentas) {
00374         return studentas.getGalutinis() >= 5; // Use getter
00375     });
00376
00377     // Kopijuojame "vargšyku" į naują konteinerį
00378     vargsiukai.insert(vargsiukai.end(), it, studentai.end());
00379
00380     // Pašaliname "vargšyku" iš pradinio konteinerio
00381     studentai.erase(it, studentai.end());
00382
00383     // Įrašome "vargšyku" į failą
00384     ofstream outVargsiukai("stud_b.txt");
00385     if (!outVargsiukai.is_open()) {
00386         throw runtime_error("Nepavyko atidaryti failo");
00387     }
00388     outVargsiukai << left << setw(15) << "Vardas" << setw(20) << "Pavarde" << "Galutinis" << endl;
00389     for (const auto& studentas : vargsiukai) {
00390         outVargsiukai << left << setw(15) << studentas.getVardas() << setw(20) << studentas.getPavarde()
00391             << fixed << setprecision(2) << studentas.getGalutinis() << endl;
00392     }
00393     outVargsiukai.close();
00394
00395     // Įrašome "kietiakus" į failą
00396     ofstream outKietiakai("stud_g.txt");
00397     if (!outKietiakai.is_open()) {
00398         throw runtime_error("Nepavyko atidaryti failo");
00399     }
00400     outKietiakai << left << setw(15) << "Vardas" << setw(20) << "Pavarde" << "Galutinis" << endl;
00401     for (const auto& studentas : studentai) {
00402         outKietiakai << left << setw(15) << studentas.getVardas() << setw(20) << studentas.getPavarde()
00403             << fixed << setprecision(2) << studentas.getGalutinis() << endl;
00404     }
00405     outKietiakai.close();
00406

```

```

00407     auto end = high_resolution_clock::now();
00408     auto duration = duration_cast<milliseconds>(end - start);
00409     cout << "Studentu skaidymas ir irasymas i failus uztruko: " << duration.count() << " ms" << endl;
00410 }
00411
00412
00413 void FailuGeneravimas(int studentuSk, int pazymiuSk);
00414 void Test1();
00415 void Test2();
00416 void Test3();
00417 void TestStudentas();

```

6.2 mano_lib.h

```

00001 #include <iomanip>
00002 #include <iostream>
00003 #include <string>
00004 #include <vector>
00005 #include <algorithm>
00006 #include <cstdlib>
00007 #include <ctime>
00008 #include <fstream>
00009 #include <chrono>
00010 #include <stdexcept>
00011 #include <sstream>
00012 #include <deque>
00013 #include <list>
00014 #include <numeric>
00015 #include <limits>
00016 #include <exception>
00017 #include <variant>
00018 #include <cassert>
00019
00020 using std::deque;
00021 using std::list;
00022 using std::min;
00023 using std::istringstream;
00024 using std::ostringstream;
00025 using std::stringstream;
00026 using std::cin;
00027 using std::cout;
00028 using std::endl;
00029 using std::string;
00030 using std::vector;
00031 using std::fixed;
00032 using std::setprecision;
00033 using std::sort;
00034 using std::to_string;
00035 using std::rand;
00036 using std::srand;
00037 using std::time;
00038 using std::copy;
00039 using std::ifstream;
00040 using std::ofstream;
00041 using std::numeric_limits;
00042 using std::streamsize;
00043 using std::ios;
00044 using std::left;
00045 using std::setw;
00046 using std::chrono::high_resolution_clock;
00047 using std::chrono::duration_cast;
00048 using std::chrono::milliseconds;
00049 using std::chrono::seconds;
00050 using std::runtime_error;
00051 using std::invalid_argument;
00052 using std::cerr;
00053 using std::exception;
00054 using std::move;
00055 using std::accumulate;
00056 using std::max;
00057 using std::is_same;
00058 using std::ostream;
00059 using std::istream;
00060 using std::variant;

```

Index

ObjPirmaUzduotis, [1](#)

spausdinti

Studentas, [12](#)

Studentas, [11](#)

spausdinti, [12](#)

Zmogus, [12](#)